

“TUGAS PRATIKUM 5 SISTEM OPERASI”

**Disusun Untuk Memenuhi Tugas Mata Kuliah
Sistem Operasi**

Dosen : Triawan Adi Cahyanto, M.Kom



Disusun Oleh:

NAMA : Reandra Khansa Faadihilah

NIM : 2410651060

KELAS : TI – C

Tanggal : 10-11-2025

UNIVERSITAS MUHAMMADIYAH JEMBER

2025-2026

DAFTAR ISI

DAFTAR ISI	2
Dining Philosophers Problem.....	3
Program Menyebabkan Deadlock	4
SOLUSI A : Resource Ordering (Pengurutan).....	7
SOLUSI B : Limiting Philosophers.....	10
KESIMPULAN	12

Dining Philosophers Problem

Pada tugas kali ini kita membuat sebuah program yang bisa menyebabkan deadlock dan memberikan 2 solusi, pertama yaitu dengan Ordering (Pengurutan), kedua Limit 4 filsuf.

PERSIAPAN

```
^ ~ > cd S05
^ ~/S05 > ls
pratikum-sinkronisasi
^ ~/S05 > cd pratikum-sinkronisasi/
^ ~/S05/pratikum-sinkronisasi > ls
percobaan1 percobaan2 percobaan3 percobaan4 percobaan5
^ ~/S05/pratikum-sinkronisasi > cd percobaan5
^ S05/pratikum-sinkronisasi/percobaan5 > |
```

Membuat folder untuk pratikum

- mkdir pratikum-sinkronisasi

Jika sudah membuat tinggal mengakses folder saja dengan :

- cd pratikum-sinkronisasi/

Setelah membuat folder kita membuat sub folder lagi di Dalamnya dengan nama percobaan5

- mkdir percobaan5

Program Menyebabkan Deadlock

Pada topik ini membuat sebuah program yang bisa menyebabkan deadlock dimana ada 5 filsuf yang duduk melingkar di meja makan, di antara setiap 2 filsuf tersebut terdapat 1 sumpit (total 5 sumpit) setiap filsuf membutuhkan 2 sumpit untuk makan (Kiri dan Kanan) Hal ini bisa menyebabkan filsuf bertindak bersamaan tanpa koordinasi akibatnya tidak ada yang bisa makan karena semua menunggu satu sama lain.

PRAKTIK

Membuat file deadlock_filsuf.c

^ S05/pratikum-sinkronisasi/percobaan5 > gedit deadlock_filsuf.c

```
1 #include <stdio.h>
2 #include <pthread.h>
3 #include <unistd.h>
4
5 #define NUM_PHILOSOPHERS 5
6
7 pthread_mutex_t chopstick[NUM_PHILOSOPHERS];
8
9 void* philosopher(void* num) {
10     int id = *(int*)num;
11
12     while(1) {
13         printf("Filsuf-%d sedang Berpikir... \n", id);
14         sleep(1);
15
16         // Ambil sumpit kiri dan kanan (urutan sama + deadlock!)
17         printf("Filsuf-%d lapar, mencoba ambil sumpit... \n", id);
18         pthread_mutex_lock(&chopstick[id]);
19         printf("Filsuf-%d mengambil sumpit kiri (%d)\n", id, id);
20
21         sleep(1); // menunggu sumpit kanan
22
23         pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
24         printf("Filsuf-%d mengambil sumpit kanan (%d)\n", id, (id + 1) % NUM_PHILOSOPHERS);
25
26         printf("Filsuf-%d sedang MAKAN... \n", id);
27         sleep(2);
28
29         // Letakkan Kembali sumpit
30         pthread_mutex_unlock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
31         pthread_mutex_unlock(&chopstick[id]);
32     }
33 }
34
35 int main() {
36     pthread_t thread[NUM_PHILOSOPHERS];
37     int ids[NUM_PHILOSOPHERS];
38
39     printf("=====\n");
40     printf("Program Menyebabkan Deadlock\n");
41     printf("Tugas Praktikum - 5 Synchronization Sistem\n");
42     printf("Nama : Reandra Khansa Faadinilah\n");
43     printf("NIM : 2410651060\n");
44     printf("Welas: TI - C\n");
45     printf("=====\n\n");
46
47     for (int i = 0; i < NUM_PHILOSOPHERS; i++)
48         pthread_mutex_init(&chopstick[i], NULL);
49
50     for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
51         ids[i] = i;
52         pthread_create(&thread[i], NULL, philosopher, &ids[i]);
53     }
54
55     for (int i = 0; i < NUM_PHILOSOPHERS; i++)
56         pthread_join(thread[i], NULL);
57
58     return 0;
59 }
```

COMPILE PROGRAM DAN OUTPUTNYA :

```
^ S05/pratikum-sinkronisasi/percobaan5 > gcc deadlock_filsuf.c -o deadlock_filsuf
./deadlock_filsuf 60

Program Menyebabkan Deadlock
Tugas Praktikum - 5 Synchronization Sistem
Nama : Reandra Khansa Faadihilah
NIM : 2410651060
Kelas: TI - C

Filsuf-0 sedang berpikir ...
Filsuf-1 sedang berpikir ...
Filsuf-2 sedang berpikir ...
Filsuf-3 sedang berpikir ...
Filsuf-4 sedang berpikir ...
Filsuf-1 lapar, mencoba ambil sumpit ...
Filsuf-1 mengambil sumpit kiri (1)
Filsuf-4 lapar, mencoba ambil sumpit ...
Filsuf-4 mengambil sumpit kiri (4)
Filsuf-0 lapar, mencoba ambil sumpit ...
Filsuf-0 mengambil sumpit kiri (0)
Filsuf-2 lapar, mencoba ambil sumpit ...
Filsuf-2 mengambil sumpit kiri (2)
Filsuf-3 lapar, mencoba ambil sumpit ...
Filsuf-3 mengambil sumpit kiri (3)
^C
^ S05/pratikum-sinkronisasi/percobaan5 > |
```

c -gcc © 19:58

PENJELASAN

Mengapa deadlock bisa terjadi?

Deadlock terjadi karena ketiga kondisi :

1. Mutual Exclusion

```
pthread_mutex_t chopstick[NUM_PHILOSOPHERS];
```

mutual exclusion lock, artinya sumpit hanya bisa dipegang oleh satu filsuf pada satu waktu.

```
pthread_mutex_lock()
```

Ketika dipanggil filsuf lain tidak bisa mengambil sumpit sampai di lepaskan dengan **pthread_mutex_unlock()**

2. Hold and Wait

Proses menahan sumber daya dan menunggu yang lain

```
pthread_mutex_lock(&chopstick[id]);  
  
sleep(1);  
  
pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
```

Setelah filsuf mengambil sumpit kiri (**pthread_mutex_lock(&chopstick[id])**) dia tidak langsung melepaskannya. Ia menunggu sumpit kanan di baris berikutnya. Jadi saat baris **sleep(1)** dan **pthread_mutex_lock()** kedua dieksekusi, ia memegang satu sumber daya sambil menunggu lainnya.

3. Circular Wait

Semua proses menunggu sumber daya yang dipegang oleh proses lain membentuk lingkaran..

```
pthread_mutex_lock(&chopstick[id]);  
  
pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
```

Semua filsuf menjalankan code nya bersamaan sehingga menyebabkan :

1. Filsuf-0 mengunci sumpit[0], menunggu sumpit[1]
2. Filsuf-1 mengunci sumpit[1], menunggu sumpit[2]
... dan seterusnya, hingga
3. Filsuf-4 mengunci sumpit[4], menunggu sumpit[0].

Lingkaran ini tidak bisa diputus, sehingga mereka saling menunggu selamanya.

SOLUSI A : Resource Ordering (Pengurutan)

Pada topik ini kita mengatasi terjadinya deadlock dengan dengan mengubah urutan pengambilan sumpit secara bergantian, filsuf bisa makan bergantian tanpa saling menunggu tidak ada deadlock, dan semua mendapat kesempatan makan.

PRAKTIK

Membuat file dengan nama solusi_deadlock_urut.c dengan text editor gedit

```
#include <stdio.h>
#include <pthread.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t chopstick[NUM_PHILOSOPHERS];
int eat_count[NUM_PHILOSOPHERS] = {0};

void* philosopher(void* num) {
    int id = *(int*)num;

    while(1) {
        printf("Filsuf-%d Berpikir ... \n", id);
        sleep(1);

        if (id % 2 == 0) {
            pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
            pthread_mutex_lock(&chopstick[id]);
        } else {
            pthread_mutex_lock(&chopstick[id]);
            pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
        }

        printf("Filsuf-%d makan ... \n", id);
        eat_count[id]++;
        sleep(2);

        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
    }
}

int main() {
    pthread_t thread[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    printf("=====\n");
    printf("TUGAS PRATIKUM 5\n");
    printf("Solusi deadlock dengan Pengurutan\n");
    printf("Nama : Reandra Khansa Faadilah\n");
    printf("Nim : 2410651060\n");
    printf("Kelas : TI - C\n");
    printf("=====\n");

    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_init(&chopstick[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &ids[i]);
    }

    sleep(60); // simulasi selama 60 detik

    printf("\n== Statistik (60 detik) ==\n");
    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        printf("Filsuf-%d makan %d kali\n", i, eat_count[i]);

    return 0;
}
```

COMPILE FILE DAN OUTPUTNYA

```
(org.gnome.gedit:77900): libenchant-WARNING **: 20:03:41.972: broker.vala:156: Error loading plugin:
^ S05/pratikum-sinkronisasi/percobaan5 > gcc solusi_deadlock_urut.c -o solusi_deadlock_urut
^ S05/pratikum-sinkronisasi/percobaan5 > ./solusi_deadlock_urut
```

```
=====
TUGAS PRATIUM 5
Solusi deadlock dengan Pengurutan
Nama : Reandra Khansa Faadihilah
Nim : 2410651060
Kelas : TI - C
=====
```

```
Filsuf-0 berpikir ...
Filsuf-1 berpikir ...
Filsuf-2 berpikir ...
Filsuf-3 berpikir ...
Filsuf-4 berpikir ...
Filsuf-1 makan ...
Filsuf-4 makan ...
Filsuf-4 berpikir ...
```

```
=== Statistik (60 detik) ===
Filsuf-0 makan 12 kali
Filsuf-1 makan 12 kali
Filsuf-2 makan 11 kali
Filsuf-3 makan 10 kali
Filsuf-4 makan 15 kali
^ S05/pratikum-sinkronisasi/percobaan5 > |
```

PENJELASAN :

Pada program ini mengurutkan pengambilan sumpit secara bergantian, sehingga filsuf bisa makan bergantian tanpa saling tunggu, sehingga semua mendapatkan kesempatan untuk makan

```
if (id % 2 == 0) {

    pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]); // kanan dulu

    pthread_mutex_lock(&chopstick[id]);

} else {

    pthread_mutex_lock(&chopstick[id]);

    pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);

}
```


Pada baris `if (id % 2 == 0)` mengubah urutan pengambilan:

Filsuf genap: ambil kanan $\rightarrow$ kiri.

Filsuf ganjil: ambil kiri $\rightarrow$ kanan.

Tidak ada lagi lingkaran sempurna, jadi tidak ada circular wait.

SOLUSI B : Limiting Philosophers

Pada topik ini kita mencegah deadlock dengan cara membatasi jumlah filsuf yang dapat makan secara bersamaan menggunakan semaphore, tanpa adanya semaphore dapat memicu terjadinya deadlock

PRAKTIK

Membuat file solusi_deadlock_limit.c dengan text editor gedit

```
^ S05/pratikum-sinkronisasi/percobaan5 > gedit solusi_deadlock_limit.c
```

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define NUM_PHILOSOPHERS 5

pthread_mutex_t chopstick[NUM_PHILOSOPHERS];
sem_t room; // batasan jumlah filsuf
int eat_count[NUM_PHILOSOPHERS] = {0};

void* philosopher(void* num) {
    int id = *(int*)num;

    while(1) {
        printf("Filsuf-%d berpikir... \n", id);
        sleep(1);

        sem_wait(&room); // izin masuk ruang makan

        pthread_mutex_lock(&chopstick[id]);
        pthread_mutex_lock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);

        printf("Filsuf-%d makan... \n", id);
        eat_count[id]++;
        sleep(2);

        pthread_mutex_unlock(&chopstick[id]);
        pthread_mutex_unlock(&chopstick[(id + 1) % NUM_PHILOSOPHERS]);
        sem_post(&room);
    }
}

int main() {
    pthread_t thread[NUM_PHILOSOPHERS];
    int ids[NUM_PHILOSOPHERS];

    printf("=====\n");
    printf("Solusi Deadlock Limit 4 filsuf\n");
    printf("Nama : Reandra Mnansa Faadinlah\n");
    printf("Nim : 2410651060\n");
    printf("Melas : TI - C\n");
    printf("=====\n");

    sem_init(&room, 0, 4); // hanya 4 filsuf boleh makan bersamaan
    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        pthread_mutex_init(&chopstick[i], NULL);

    for (int i = 0; i < NUM_PHILOSOPHERS; i++) {
        ids[i] = i;
        pthread_create(&thread[i], NULL, philosopher, &ids[i]);
    }

    sleep(60);

    printf("\n== Statistik (60 detik) ==\n");
    for (int i = 0; i < NUM_PHILOSOPHERS; i++)
        printf("Filsuf-%d makan %d kali\n", i, eat_count[i]);

    return 0;
}
```

COMPILE DAN OUTPUNYA :

```
(org.gnome.gedit:95713): libenchant-WARNING **: 20:17:36.173: broker.vala:156: Error loading plugin:  
^ S05/pratikum-sinkronisasi/percobaan5 > gcc solusi_deadlock_limit.c -o solusi_deadlock_limit  
^ S05/pratikum-sinkronisasi/percobaan5 > ./solusi_deadlock_limit
```

```
=====
Solusi Deadlock Limit 4 filsuf
Nama : Reandra Khansa Faadihlah
Nim : 2410651060
Kelas : TI - C
=====
Filsuf-0 berpikir ...
Filsuf-4 berpikir ...
Filsuf-3 berpikir ...
Filsuf-1 berpikir ...
Filsuf-2 berpikir ...
Filsuf-4 makan ...
```

```
=== Statistik (60 detik) ===
Filsuf-0 makan 12 kali
Filsuf-1 makan 12 kali
Filsuf-2 makan 12 kali
Filsuf-3 makan 12 kali
Filsuf-4 makan 12 kali
^ S05/pratikum-sinkronisasi/percobaan5 > |
```

PENJELASAN :

sem_t room

Semaphore yang berfungsi sebagai “Izin Masuk ruang makan”

sem_wait(&room);

Membatasi akses masuk ke ruang makan

1. Ketika seorang filsuf ingin makan, dia harus memanggil `sem_wait(&room)`.
2. Jika masih ada izin (nilai semaphore > 0), dia bisa lanjut makan.
3. Jika izin habis (nilai semaphore = 0), dia harus menunggu sampai ada filsuf lain yang selesai makan dan melepaskan izin (`sem_post(&room)`).

sem_t room

- Setelah selesai makan, filsuf melepaskan satu izin.
- Nilai semaphore bertambah 1 filsuf lain yang menunggu bisa masuk bergantian.

KESIMPULAN

Dari ketiga program yang diuji, dapat disimpulkan bahwa deadlock terjadi karena tidak adanya pengaturan urutan atau batasan dalam penggunaan sumber daya Bersama, Solusi dengan Resource Ordering mencegah circular wait, sedangkan solusi dengan Semaphore (Limiting Philosophers) memberikan pencegahan yang lebih efektif sekaligus menjaga keadilan (fairness) dan menghilangkan risiko starvation, Praktikum ini membuktikan pentingnya penerapan sinkronisasi dan koordinasi proses agar sistem berjalan aman, efisien, dan bebas kebuntuan.